ERHVERVSAKADEMI
KØBENHAVN

Databases and Requests

PBA Cybersikkerhed

lavr@ek.dk
lira@ek.dk

Agenda

- Databases
  - SQLite
  - SQL Connector
- Requests
- Mandatory



Databases

Back to Agenda

ERHVERVSAKADEMI
KØBENHAVN

Databases

Almost every website and app stores its data in a database. The database lives on the backend – the user never talks to it directly.

Frontend

What the user sees



request

page

Backend

Your Python code

```
c.execute(
  "SELECT * FROM user
   WHERE first_name = ?",
  (name,))
```

SQL

rows

Database

Stores data in tables

```
user_id  first_name
1        Alice
2        Bob
3        Charlie
```

Focus: how to connect to and use a database from Python – not how it works inside.

https://www.geeksforgeeks.org/python/introduction-to-sqlite-in-python/

SQL, SQLite and sqlite3

Three names that are easy to mix up:

SQL

The language – the commands you write to talk to a database.

```
SELECT * FROM user;
```

SQLite

The database – the whole thing is one file. No server needed.

```
# one file on your disk
test.db
```

sqlite3

The Python module – built in, nothing to install.

```
import sqlite3
sqlite3.connect("test.db")
```

The SQL commands you will use

CREATE TABLE	INSERT	SELECT	UPDATE	DELETE
make a table	add rows	read rows	change rows	remove rows

https://docs.python.org/3/library/sqlite3.html

ERHVERVSAKADEMI KØBENHAVN

Databases – connection strings

A connection string is one or several strings, that includes all the information needed to connect to the server. When we work with a local database, we simply need to point it to our database file as such:

```
import sqlite3

conn = sqlite3.connect("test.db")
```

The database file will be created if it does not exist. In this case our connection string is simply `"test.db"`.

A more advanced connection string to a remote connection could look like this:

```
SQL_CONNECTION_STRING="Driver={ODBC Driver 18 for SQL
Server};Server=123.132.33.44;Database={clients};Encrypt=yes;TrustServerCertificate=no;A
uthentication=ActiveDirectoryInteractive"
```

https://www.geeksforgeeks.org/python/introduction-to-sqlite-in-python/

Databases – connection strings continued

Since our database is local, we can combine it with a Path object as you already know:

```
database_path = Path.cwd().joinpath("test.db")

conn = sqlite3.connect(database_path)
```

Every type of database has their own syntax for how a connection string should be formatted, and some modules help you make this string as part of the function call. You should always check the documentation for the module you are using, to get the right syntax.

https://www.geeksforgeeks.org/python/introduction-to-sqlite-in-python/

Databases – cursor

Now that we have connected to our database, it's time to make the cursor. A cursor is a type of object that executes our queries on the database.

Once you have a connection to the database established, it is simple to create the cursor object as such:

```
conn = sqlite3.connect(database_path)
c = conn.cursor()
```

We can check that "c" currently represents our cursor object:

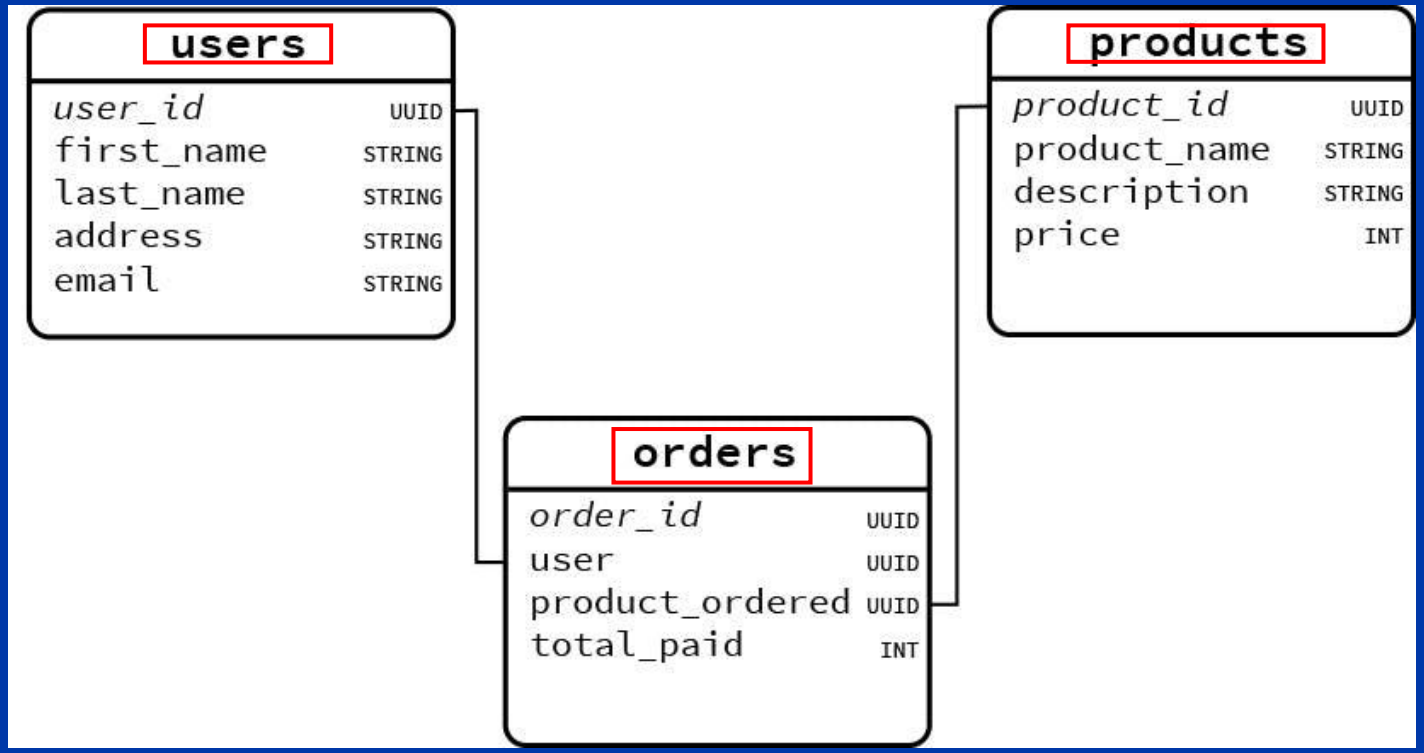
```
print(c) # <sqlite3.Cursor object at 0x0000017F131EA1C0>
```

Now we are ready to specify queries to the cursor, that will be executed in the database.

https://www.geeksforgeeks.org/python/introduction-to-sqlite-in-python/

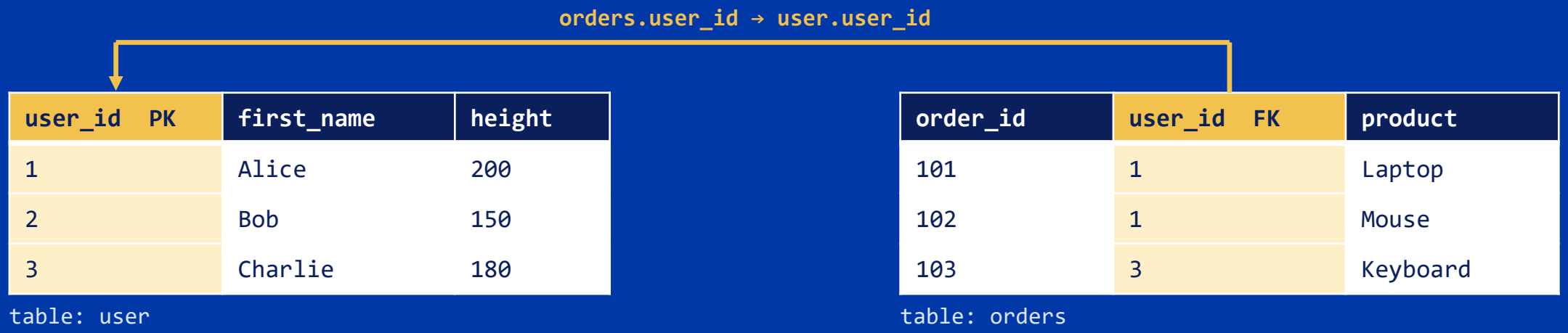
Database – Creating tables

Our database is empty and devoid of structure; we want to create some tables that can store our data. We define the columns/headers of each table, and the rows represent the entries.



https://www.cockroachlabs.com/blog/database-schema-beginners-guide/

Primary keys and foreign keys



user_id PK	first_name	height
1	Alice	200
2	Bob	150
3	Charlie	180

table: user

order_id	user_id FK	product
101	1	Laptop
102	1	Mouse
103	3	Keyboard

table: orders

Primary key (PK)

Uniquely identifies each row – never duplicated, never empty.

```
user_id INTEGER PRIMARY KEY
```

Foreign key (FK)

Points to a primary key in another table: order 101 belongs to user 1 (Alice).

```
FOREIGN KEY (user_id)
  REFERENCES user(user_id)
```

https://www.sqlite.org/foreignkeys.html

ERHVERVSAKADEMI KØBENHAVN

Database – Creating tables continued

To create a table, we create a query that we then send to our cursor object.

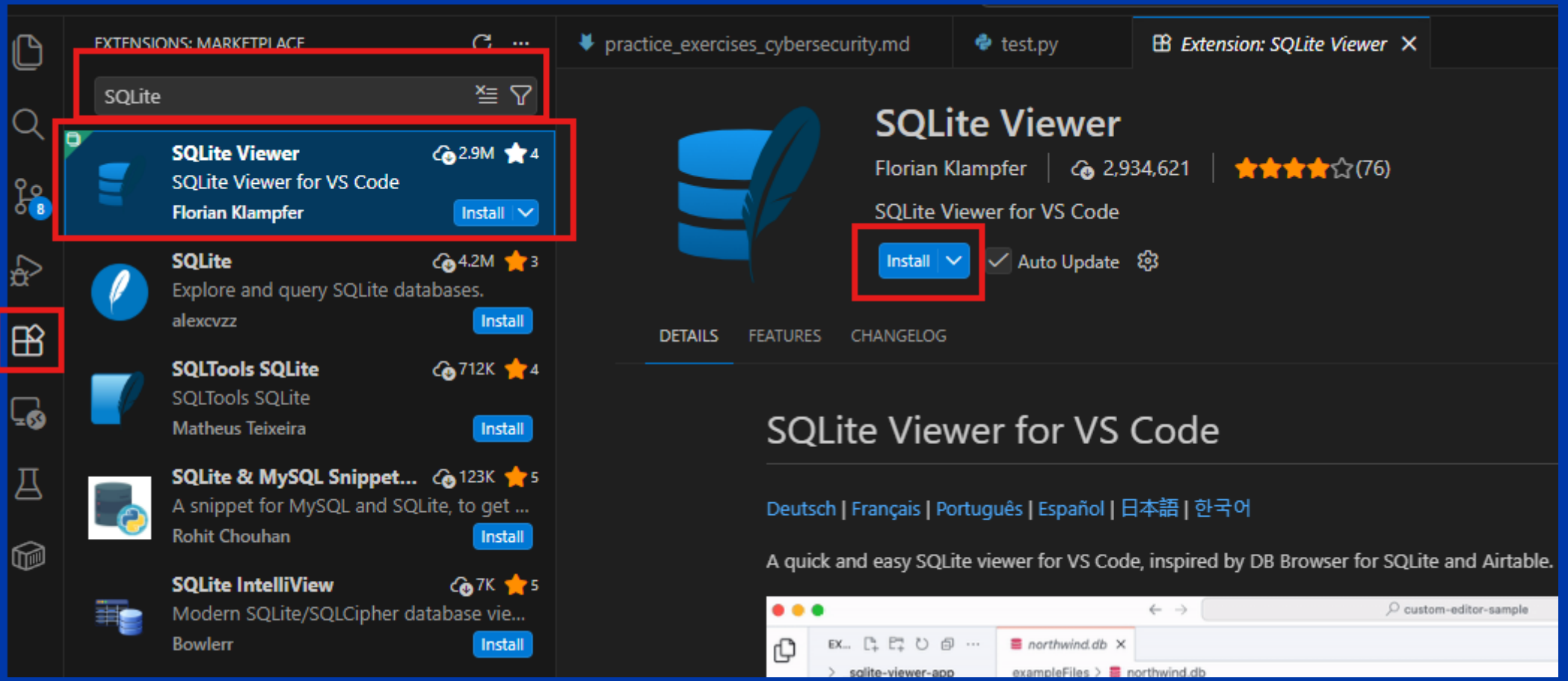
```
query = '''CREATE TABLE IF NOT EXISTS user(
            user_id INTEGER PRIMARY KEY NOT NULL,
            first_name TEXT NOT NULL,
            height INTEGER
            )
            '''

c.execute(query)
```

We have now made a table called “user” and the columns “user_id”, “first_name” and “height” and given them appropriate datatypes.

https://docs.python.org/3/library/sqlite3.html

Install SQLite Viewer extension



ERHVERVSAKADEMI KØBENHAVN

Database – Inserting data

Now that we have a table, we can insert “rows” into the table. These are our data entries.

If I want to make an entry I could do as such:

```
c.execute ("INSERT INTO user VALUES (1, ‘Alice', '200')")
```

If I want to follow good practice, I include the column names as part of my query but this is not required:

```
c.execute("INSERT INTO user (user_id, first_name, height) VALUES (2, 'Bob', '150')")
```

The INSERT statement opens a transaction that needs to be committed. To commit this I can call the commit function on the connection object as such:

```
conn.commit()
```

If I have nothing more to insert for now, it is good practice to close the connection:

```
conn.close()
```

https://docs.python.org/3/library/sqlite3.html

Inserting Rows with Prepared Statements

Instead of placing values directly into the query string, we use **prepared statements** – a safer and cleaner approach.

To insert a row, we pass **?** as placeholders and provide the values as a separate tuple:

```
c.execute("INSERT INTO user VALUES (?, ?, ?)", (3, 'Charlie', ‘180’))
```

Or

```
c.execute("INSERT INTO user (user_id, first_name, height) VALUES (?, ?, ?)", (4, ‘Dave', ‘170'))
```

Why prepared statements? SQL injection

A user types into the login: Username `admin' --` Password `123`

✗ Unsafe: f-string

```
query = f"SELECT * FROM user WHERE
  username = '{u}' AND password = '{p}'"
c.execute(query)
```

The database receives:

```
SELECT * FROM user WHERE
username = 'admin' --' AND password = '123'
```

-- comments out the rest → **logged in as admin!**

✓ Safe: prepared statement

```
query = "SELECT * FROM user WHERE
  username = ? AND password = ?"
c.execute(query, (u, p))
```

The database receives:

```
SQL:    ... WHERE username = ? AND password = ?
values: "admin' --", "123"
```

The input is only data → **login fails.**

Same rule as last time: never trust user input. Use ? for every value.

https://owasp.org/www-community/attacks/SQL_Injection

ERHVERVSAKADEMI KØBENHAVN

My first database

1. Create a local database using the “**sqlite3**” module.
2. Create a table called **students** with at least these columns:

Column	Type	Constraint
student_id	INTEGER	PRIMARY KEY
first_name	TEXT	NOT NULL
grade	INTEGER	

3. Use prepared statements to insert **at least 3** student entries into your table.
4. Wrap your database operations in try/except/finally blocks:
   a. Use **except** to catch and print any database errors
   b. Use **finally** to ensure the connection is always closed, even if an error occurs

- Bonus: Add a second try/except that attempts to insert a duplicate student_id and catches the specific error that SQLite raises for unique constraint violations.

Database – reading data

Setting up our connection again

```
import sqlite3
conn = sqlite3.connect("test.db")
c = conn.cursor()
```

Now we can execute a query to get all data from a certain table.

```
data = c.execute("SELECT * FROM user ORDER BY user_id").fetchall()
```

`fetchall()` returns a list of the data gathered by the query. If there's no data it will be an empty list.

https://docs.python.org/3/library/sqlite3.html

Database – reading data continued

We could make some logic to check whether our query fetched any data.

```
if not len(data):
    print("Table is empty!")
else:
    print(data)
```

It also makes sense to include error handling, because if our query is faulty or the specified table doesn't exist – the program will crash.

https://docs.python.org/3/library/sqlite3.html

Database – deleting data

With the same setup as before and our cursor object ready, we can simply delete entries in a table by making a DELETE query.

For example:

```
c.execute("DELETE FROM user WHERE first_name='Alice'")
```

We can also delete the entire table at once just writing:

```
c.execute("DELETE FROM user")
```

Or

```
c.execute("DROP TABLE user")
```

To make sure your database is saved, it is good practice to `commit()` changes after a `DELETE`:

```
conn.commit()
```

EK ERHVERVSAKADEMI KØBENHAVN

Database – one connection, many queries

Do I need a new connection for every query? **No – one connection does it all.**

Connect

```
sqlite3.connect()
```

once – at the start

Cursor

```
conn.cursor()
```

once

Queries

```
c.execute("INSERT …")
c.execute("UPDATE …")
c.execute("SELECT …")
```

as many as you need

Commit

```
conn.commit()
```

saves the changes

Close

```
conn.close()
```

at the end

```
conn = sqlite3.connect("test.db")       # once
c = conn.cursor()                       # once
for s in students:
    c.execute("INSERT INTO students VALUES (?, ?, ?)", s)
conn.commit()                           # save the inserts
rows = c.execute("SELECT * FROM students").fetchall()
conn.close()                            # once, at the end
```

Don't forget commit()

INSERT, UPDATE and DELETE are only saved after conn.commit(). Reading with SELECT needs no commit.

https://docs.python.org/3/library/sqlite3.html

ERHVERVSAKADEMI KØBENHAVN

My first database - continued

- Using your database from before, make a program that can:

1. Read the data from a table and print it out.
2. Inform you if the table is empty.
3. Not crash if the table doesn’t exist.
4. Use a DELETE statement to remove a query.

Bonus:

- Experiment using UPDATE statements. Research the syntax for using UPDATE.
- Have your program create a table if the query tries to find a table that doesn’t exist.

Connecting to a live database (mysql-connector-python)

```
import mysql.connector

conn =
mysql.connector.connect(host="sql.nordkvist.org",user="guest",password="HJYW2F
jRqJTNIAambTb0wBV", database="ek_programming")

c = conn.cursor()

c.execute("SELECT * FROM students")

data = c.fetchall()

if not len(data):

    print("Table is empty!")

else:

    print(data)
```



Requests

Back to Agenda

ERHVERVSAKADEMI KØBENHAVN

Requests – like a browser, but in code

When you open a web page, your browser sends a request and gets a response back. With requests, Python does exactly the same – no browser needed.

CLIENT

Browser

you click

ek.dk

or

Python script

```
requests.get("https://ek.dk")
```

HTTP request **GET /**

HTTP response **200 OK + the page**

SERVER

Web server

ek.dk, GitHub, an API …

Send a request

```
requests.get(url)
requests.post(url, data=...)
```

Read the answer

```
r.status_code   # 200
r.text          # the page
```

Why is this useful in cyber security?

- Test login forms automatically
- Work with API's
- Check many websites at once

ERHVERVSAKADEMI KØBENHAVN

Requests – behind the scenes

In Python

Request

```
import requests
r = requests.get(
    "https://jsonplaceholder.typicode.com/posts/1",
    headers={"Accept": "application/json"})
```

sent →

```
GET /posts/1 HTTP/1.1
Host: jsonplaceholder.typicode.com
User-Agent: python-requests/2.32.3
Accept: application/json
Accept-Encoding: gzip, deflate
Connection: keep-alive
```

Response

```
r.status_code              # 200
r.headers["Content-Type"]  # "application/json; ..."
r.text                     # the body as one string
r.json()                   # the body as a dict
r.json()["title"]          # "sunt aut facere ..."
```

← parsed

```
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8
Content-Length: 292
Cache-Control: max-age=43200

{"userId": 1, "id": 1,
 "title": "sunt aut facere ...", ...}
```

On the wire – plain text over HTTP(S)

First line = method + path / status code **Headers** = key: value

https://requests.readthedocs.io/en/latest/user/quickstart/

Requests – GET

```
pip install requests
```

Syntax:

```
requests.get(url, params={key: value}, **kwargs)
```

Example:

```
import requests
response = requests.get("https://ek.dk")
print(response.status_code) # Prints 200 if things go well
print(response.text) # Prints the content of the response in unicode
print(response.content) # Prints the content of the response in bytes
```

https://www.geeksforgeeks.org/python/python-requests-tutorial/

Requests - continued

In the HTTP protocol, we have different methods. Most of the time we are operating with GET or POST, but the requests library supports them all.

Method	Description
GET	Retrieve information from the server
POST	Send data to the server to create/update resources
PUT	Replace the target resource with new data.
DELETE	The DELETE method deletes the specified resource
HEAD	Retrieve headers only (no body)
PATCH	Apply partial modifications to a resource

https://www.geeksforgeeks.org/python/python-requests-tutorial/

Requests – POST

We can define our payload or “body” of the request and attach it to our call. The requests library will automatically structure the request for us.

```
import requests

payload = {'username': 'alice', 'password': 'strongpassword'}
response = requests.post("https://httpbin.org/post", data=payload)
print(response.text)
```

https://www.geeksforgeeks.org/python/python-requests-tutorial/

Requests – authentication

The requests library supports many types of authentication. You can find some examples of authentication on the following link.

Different methods require more or less complex ways, and if you want to work with something such as OAuth2, you might want to look into a library especially suited for that purpose.

Here's an example of using HTTPBasicAuth with the requests library:

```
import requests

from requests.auth import HTTPBasicAuth

response = requests.get('https://api.github.com/user', auth=HTTPBasicAuth('Alice', 'strongpassword'))

print(response.status_code)
```

Postman

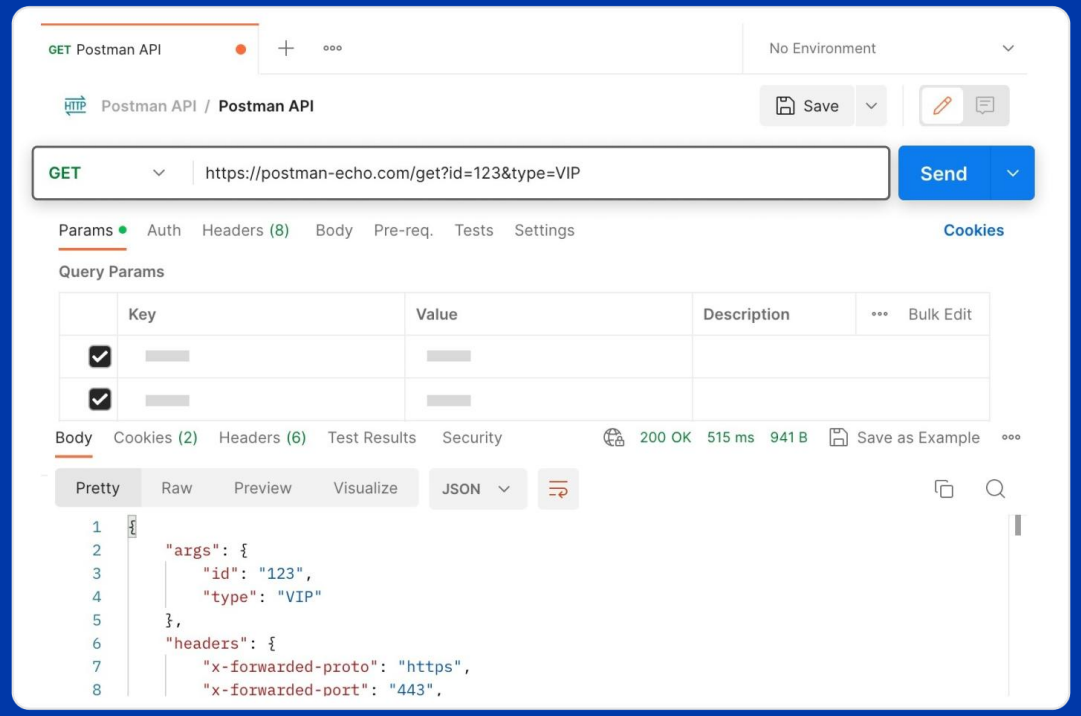
Postman is a free tool for sending HTTP requests through a graphical interface – no code needed.

You set the same things as with the requests library: the **method**, **URL**, **parameters**, **headers**, **authentication** and **body**.

The response is shown right away, so you can explore an API before writing any Python.

Postman can also generate the Python code for a request.

Download: postman.com/downloads



ERHVERVSAKADEMI KØBENHAVN

Exploring an API with Postman

Before writing any Python, send the requests by hand in Postman and look at what comes back.

Requirements

1. Send a **GET** request to https://jsonplaceholder.typicode.com/posts/1
   a. Find the status code
   b. Find the Content-Type header under **Headers**
   c. Find the title in the response body
2. Change the URL to /posts/9999. Which status code do you get now – and what does it mean?
3. Send a **POST** request to https://jsonplaceholder.typicode.com/posts with this JSON body (**Body → raw → JSON**):
   `{"title": "Hello", "body": "My first post", "userId": 1}`
   Which status code and body do you get back?
4. Click the **Code** button </> and choose **Python – Requests**. Compare it with the code you would write yourself.

Bonus

- Under **Authorization**, choose **Basic Auth** and type a username and password. Which header does Postman add to the request?

My first requests

Use the **requests** library from Python to get data from an API.

Requirements

1. Activate your venv and install requests (if you haven't already):
   `pip install requests`
2. Write Python code that:
   a. Makes a GET request to https://jsonplaceholder.typicode.com/users/1
   b. Prints the status code
   c. Prints the Content-Type header of the response
   d. Only if the status code is 200: prints the user's **name** and **email**
   e. Otherwise prints an error message

Expected output

```
Status code: 200
Content-Type: application/json; charset=utf-8
Leanne Graham – Sincere@april.biz
```

Hints

- response.status_code gives the status, response.headers["Content-Type"] a header
- response.json() turns the body into a dictionary
- Bonus: change the URL to /users/999 – does your error message show?

Getting to know requests

Make a program that uses the requests library to access:

https://testpages.eviltester.com/styled/auth/basic-auth-test.html

Read the content of the page and find the “protected URL”.

Make your program access the protected URL, by setting a Basic Authentication Header as part of your request.

The response should contain “You were Authenticated” if you have succeeded in authenticating.

Bonus: Make your program parse the values (username and password) from the response of your first request, and use them in your second requests to authenticate.

Requests – session objects

When we establish session in the browser, this is often done through cookies. We can create a session object to maintain a cookie across several requests.

```
import requests

session = requests.Session()

session.get('https://httpbin.org/cookies')

response = session.get('https://httpbin.org/cookies')
print(response.text)
```

https://www.geeksforgeeks.org/python/python-requests-tutorial/

Requests – best practices

- When connecting to live data, use error handling where appropriate. Examples are given in the links below.
- Maintain your sessions when possible, to minimise resources required from the server.

https://www.geeksforgeeks.org/python/python-requests-tutorial/

Mandatory – Part 2